MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

Department of Electronics and Communication Engineering



LAB FILE

SIGNALS & SYSTEMS

(ECC – 259)

Semester 3rd

Rishi

Submitted By	Submitted To
Rishi Parkash Verma	**Dr.**
03315002825	Professor
ECE-1 (3rd Semester)	Department Of

INDEX

Exp no.	Date	Exp name	Page no.	Teacher's sign	Remarks

Experiment : 1

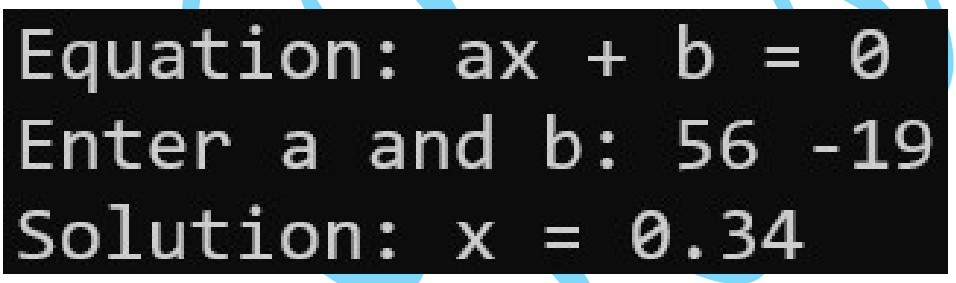
Linear equation in 1 variable

Code:

```
#include <stdio.h>

int main() {
  float a, b, x;
  printf("Equation: ax + b = 0\n");
  printf("Enter a and b: ");
  scanf("%f %f", &a, &b);
  if (a == 0) {
    printf("Not a linear equation (a=0).\n");
  } else {
    x = -b / a;
    printf("Solution: x = %.2f\n", x);
  }
  return 0;
}
```

Output:



Experiment : 2

Linear equation in 2 variable

Code:

```
#include <stdio.h>

int main() {
  float a1, b1, c1, a2, b2, c2, x, y, det;
  printf("Write two Equations in ax + by = c form\n");
  printf("Enter a1, b1, c1: ");
  scanf("%f %f %f", &a1, &b1, &c1);
  printf("Enter a2, b2, c2: ");
  scanf("%f %f %f", &a2, &b2, &c2);
  printf("Equations are:\n");
  printf("%.2fx %c %.2fy = %.2f\n", a1, b1 >= 0 ? '+' : '-', b1 >= 0 ? b1 :
  b1, c1);
  printf("%.2fx %c %.2fy = %.2f\n", a2, b2 >= 0 ? '+' : '-', b2 >= 0 ? b2 :
  b2, c2);
  //cramer's rule
  det = a1 * b2 - a2 * b1;
  if (det == 0) {
    printf("No unique solution exists.\n");
  } else {
    x = (c1 * b2 - c2 * b1) / det;
    y = (a1 * c2 - a2 * c1) / det;
    printf("Solution: x = %.2f, y = %.2f\n", x, y);
  }
  return 0;
}
```

Output:

```
Write two Equations in ax + by = c form
Enter a1, b1, c1: 1 -2 -3
Enter a2, b2, c2: 2 4 -5
Equations are:
1.00x - 2.00y = -3.00
2.00x + 4.00y = -5.00
Solution: x = -2.75, y = 0.12
```

Experiment : 3

Quadratic Equation

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
  float a, b, c, Det, r1, r2, realPart, imagPart;
  printf("Equation: ax^2 + bx + c = 0\n");
  printf("Enter a, b, c: ");
  scanf("%f %f %f", &a, &b, &c);
  printf("%.2fx^2 %c %.2fx %c %.2f = 0\n", a,
    b >= 0 ? '+' : '-', b >= 0 ? b : -b,
    c >= 0 ? '+' : '-', c >= 0 ? c : -c);

  Det = b * b - 4 * a * c;

  if (Det > 0) {
    r1 = (-b + sqrt(Det)) / (2 * a);
    r2 = (-b - sqrt(Det)) / (2 * a);
    printf("Roots are real and distinct: %.2f and %.2f\n", r1, r2);
  } else if (Det == 0) {
    r1 = -b / (2 * a);
    printf("Roots are real and equal: %.2f and %.2f\n", r1, r1);
  } else {
    realPart = -b / (2 * a);
    if (fabs(realPart) < 0.001)
      realPart = 0;
    imagPart = fabs(sqrt(-Det) / (2 * a));
    printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi\n", realPart,
    imagPart, realPart, imagPart);
  }
  return 0;
}
```

Output:

```
Equation: ax^2 + bx + c = 0
Enter a, b, c: 1 -5 6
1.00x^2 - 5.00x + 6.00 = 0
Roots are real and distinct: 3.00 and 2.00
```

```
Equation: ax^2 + bx + c = 0
Enter a, b, c: -1 0 -1
-1.00x^2 + 0.00x - 1.00 = 0
Roots are complex: 0.00 + 1.00i and 0.00 - 1.00i
```

Experiment : 4

Bisection Method

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

int main() {
  float a, b, mid, root1, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Cubic equation: Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter interval for first root: ");
  if (scanf("%f %f", &a, &b) != 2 || f(a) * f(b) > 0) {
    printf("Invalid interval\n");
    return 1;
  }

  do {
    mid = (a + b) / 2;
    iter++;
    if (f(a) * f(mid) < 0)
      b = mid;
    else
      a = mid;
  } while (fabs(f(mid)) > e);

  root1 = mid;
  printf("Iterations took %d \n", iter);
  printf("First root (bisection): %.6f\n", root1);

  p = A;
  q = B + A * root1;
  r = C + q * root1;
  d = q * q - 4 * p * r;

  if (d >= 0) {
```

Sisir

```
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
  } else {
    float real = -q / (2 * p);
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }

  return 0;
}
```

Output:

```
Cubic equation: Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 1 2 3 4
Enter interval for first root: -10 10
Iterations took 21
First root (bisection): -1.650629
Second root: -0.174685 + 1.546869i
Third root: -0.174685 - 1.546869i
```

Sisi

Experiment : 5

Secant Method

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

int main() {
  float x0, x1, x2, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Cubic equation: Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter initial guesses: ");
  if (scanf("%f %f", &x0, &x1) != 2 || (f(x0) - f(x1)) == 0) {
    printf("Invalid initial guesses.\n");
    return 1;
  }

  do {
    x2 = (f(x1) * x0 - f(x0) * x1) / (f(x1) - f(x0));
    iter++;
    x0 = x1;
    x1 = x2;
  } while (fabs(f(x2)) > e);
  printf("Iterations took %d \n", iter);
  printf("First Root is : %f\n", x2);

  p = A;
  q = B + A * x2;
  r = C + q * x2;
  d = q * q - 4 * p * r;

  if (d >= 0) {
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
```

Sisir

```
  } else {
    float real = -q / (2 * p);
    if (fabs(real) < 0.000001)
      real = 0;
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }

  return 0;
}
```

Output:

```
Cubic equation: Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 2 -3 4 -5
Enter initial guesses: -10 10
Iterations took 6
First Root is : 1.371134
Second root: 0.064433 + 1.348761i
Third root: 0.064433 - 1.348761i
```

Experiment : 6

Newton Raphson Method

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

float df(float x) {
  return (3 * A * x * x) + (2 * B * x) + C;
}

int main() {
  float x0, x1, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Newton-Raphson Method for Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter initial guess: ");
  if (scanf("%f", &x0) != 1) {
    printf("Invalid initial guess.\n");
    return 1;
  }

  do {
    if (df(x0) == 0) {
      printf("Derivative is zero. Cannot continue.\n");
      return 1;
    }
    x1 = x0 - f(x0) / df(x0);
    iter++;
    x0 = x1;
  } while (fabs(f(x1)) > e);

  printf("Iterations took %d\n", iter);
  printf("First root: %.6f\n", x1);

  p = A;
  q = B + A * x1;
```

```
  r = C + q * x1;
  d = q * q - 4 * p * r;

  if (d >= 0) {
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
  } else {
    float real = -q / (2 * p);
    if (fabs(real) < e)
      real = 0;
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }
  return 0;
}
```

Output:

```
Newton-Raphson Method for Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 2 -3 4 -5
Enter initial guess: 1
Iterations took 4
First root: 1.371134
Second root: 0.064433 + 1.348761i
Third root: 0.064433 - 1.348761i
```